

[화이트햇스쿨 4기] 프로그래밍 기초 — 도전: AST 맛보기

- 이름: 이승환
- 분석 대상: `ast.json` (멘토 제공, `pycparser`로 생성한 C 소스코드의 AST)
- 작성 프로그램: `ast.c` (분석기), `restored_source.c` (복원 결과)

1. AST란 무엇인가

AST(Abstract Syntax Tree, 추상 구문 트리) 는 소스코드를 컴파일러가 이해할 수 있도록 **문법 구조를 트리(나무) 형태로 표현**한 것이다.

예를 들어 `int main() { return main1(); }` 라는 코드는 다음과 같은 트리로 표현된다.

```
FuncDef (함수 정의)
├── decl: Decl (name = "main")
│   └── type: FuncDecl
│       └── type: TypeDecl → IdentifierType(["int"]) // 리턴타입 int
└── body: Compound (함수 본문 { })
    └── Return
        └── FuncCall (main1())
```

이 트리를 JSON으로 직렬화한 것이 바로 `ast.json` 이다. 각 노드는 `"_nodetype"` 키로 종류(`FuncDef`, `Decl`, `If`, `Return` ...)를 구분하고, 자식 노드를 키-값 / 배열로 가지고 있다. 즉 **소스코드를 "데이터"처럼 다룰 수 있게** 된 것이다.

이렇게 코드를 데이터로 다루면, 코드를 직접 실행하지 않고도 "함수가 몇 개인지", "어떤 타입을 쓰는지", "버그 패턴이 있는지" 등을 **자동으로 분석**할 수 있다. (실제로 정적 분석 도구, 컴파일러, 보안 취약점 탐지기가 모두 AST를 사용한다.)

2. 분석 방법 (ast.c 설명)

멘토가 제공한 JSON 파서 라이브러리 `json_c.c` 를 사용해 `ast.json` 을 파싱하고, 트리를 따라 내려가며 필요한 정보를 추출했다.

핵심 아이디어는 다음과 같다.

추출 항목	AST에서 찾는 위치
함수 개수	최상위 <code>ext</code> 배열에서 <code>_nodetype == "FuncDef"</code> 인 노드 수
함수 이름	<code>FuncDef</code> → <code>decl</code> → <code>name</code>
리턴 타입	<code>FuncDef</code> → <code>decl</code> → <code>type(FuncDecl)</code> → <code>type</code> 를 <code>IdentifierType</code> 까지 따라감
파라미터	<code>FuncDecl</code> → <code>args(ParamList)</code> → <code>params[]</code> 의 각 <code>type</code> 과 <code>name</code>
if 개수	함수 <code>body</code> 아래를 재귀로 훑어 <code>_nodetype == "If"</code> 인 노드 수

직접 만든 핵심 함수:

- `read_file()` : `ast.json` 파일 전체를 문자열로 읽음 (라이브러리의 `json_read`는 미구현이라 직접 작성)
- `obj_get()` / `arr_get()` : 객체·배열에서 값을 조용히 꺼내는 헬퍼
- `type_to_str()` : `PtrDecl`→`TypeDecl`→`IdentifierType` 처럼 겹겹이 싸인 타입을 "`char *`" 같은 문자열로 변환
- `count_if()` : 트리를 재귀로 내려가며 if 노드를 세는 함수

```
PS D:\이승환\명지대학교\화이트햇스쿨\1주차\프로그래밍 기초 및 응용 실습 (김진영 멘토
_과제> gcc ast.c -o ast.exe -w # -w = 모든 경고 숨김
>> .\ast.exe
>>
AST 분석 결과 (ast.json)
=====
최상위 항목 수 : 55개
함수 정의 수 : 36개 <-- (1) 함수 개수
=====

번호 함수이름 리턴타입 if수 파라미터(타입 변수명)
-----
1 main int 0 void
2 my_realloc char * 0 char * old, int oldlen, int newlen
3 error void 0 void
4 takechar void 1 void
5 get_token void 7 void
6 peek int 0 char * s
7 accept int 1 char * s
8 expect void 1 char * s
9 save_int void 0 char * p, int n
10 load_int int 0 char * p
11 emit void 1 int n, char * s
12 be_push void 0 void
13 be_pop void 0 int n
14 sym_lookup int 1 char * s
15 sym_declare void 1 char * s, int type, int value
16 sym_declare_global int 1 char * s
17 sym_define_global void 1 int current_symbol
18 sym_get_value void 5 char * s
19 be_start void 0 void
20 be_finish void 0 void
21 promote void 2 int type
22 primary_expr int 9 void
23 binary1 void 0 int type
24 binary2 int 0 int type, int n, char * s
25 postfix_expr int 3 void
26 additive_expr int 2 void
27 shift_expr int 2 void
28 relational_expr int 0 void
29 equality_expr int 2 void
30 bitwise and expr int 0 void
```

3. 기본 과제 결과

`ast.exe` 실행 결과, 총 함수 개수는 36개(최상위 항목 55개 중 함수 정의 36개 + 함수 선언 19개)이다.

번호	함수이름	리턴타입	if 개수	파라미터 (타입 변수명)
1	main	int	0	void
2	my_realloc	char *	0	char * old, int oldlen, int newlen
3	error	void	0	void
4	takechar	void	1	void
5	get_token	void	7	void
6	peek	int	0	char * s
7	accept	int	1	char * s
8	expect	void	1	char * s
9	save_int	void	0	char * p, int n
10	load_int	int	0	char * p
11	emit	void	1	int n, char * s
12	be_push	void	0	void
13	be_pop	void	0	int n
14	sym_lookup	int	1	char * s
15	sym_declare	void	1	char * s, int type, int value
16	sym_declare_global	int	1	char * s
17	sym_define_global	void	1	int current_symbol
18	sym_get_value	void	5	char * s
19	be_start	void	0	void
20	be_finish	void	0	void
21	promote	void	2	int type
22	primary_expr	int	9	void
23	binary1	void	0	int type
24	binary2	int	0	int type, int n, char * s
25	postfix_expr	int	3	void
26	additive_expr	int	2	void
27	shift_expr	int	2	void
28	relational_expr	int	0	void
29	equality_expr	int	2	void
30	bitwise_and_expr	int	0	void

번호	함수이름	리턴타입	if 개수	파라미터 (타입 변수명)
31	bitwise_or_expr	int	0	void
32	expression	int	2	void
33	type_name	void	0	void
34	statement	void	8	void
35	program	void	4	void
36	main1	int	0	void

함수 이름(primary_expr, statement, sym_declare, get_token ...)으로 미루어 이 소스는 **작은 C 컴파일러**(토큰 나이저 + 파서 + 코드 생성기)임을 알 수 있다.

4. 심화 — 원본 소스코드 복원

AST에는 원본 코드의 모든 구조 정보가 들어 있으므로, 트리를 거꾸로 따라가면 원본 소스코드를 거의 그대로 복원할 수 있다. (전체 결과는 [restored_source.c](#), 약 737줄)

아래는 복원한 함수 **3개** 예시이다.

(1) main / my_realloc

```
int main()
{
    return main1();
}

char *my_realloc(char *old, int oldlen, int newlen)
{
    char *new = malloc(newlen);
    int i = 0;
    while (i <= (oldlen - 1))
    {
        new[i] = old[i];
        i = i + 1;
    }
    return new;
}
```

- **main** : 리턴타입 **int**, 파라미터 없음, if 0개 → 표의 1번과 일치.
- **my_realloc** : 리턴타입 **char ***, 파라미터 3개(**char* old, int oldlen, int newlen**) → 표의 2번과 일치. 새 메모리를 할당하고 옛 내용을 한 바이트씩 복사하는, 직접 만든 realloc이다.

(2) error

```
void error()
{
    exit(1);
}
```

- 리턴타입 `void`, 파라미터 없음, if 0개 → 표의 3번과 일치. 에러 시 프로그램을 종료한다.

(3) takechar

```
void takechar()
{
    if (token_size <= (i + 1))
    {
        int x = (i + 10) << 1;
        token = my_realloc(token, token_size, x);
        token_size = x;
    }
    token[i] = nextc;
    i = i + 1;
    nextc = getchar();
}
```

- 리턴타입 `void`, if 1개 → 표의 4번과 정확히 일치. 버퍼가 가득 차면(`if`) 크기를 늘린 뒤 현재 문자를 토큰 버퍼에 담는 함수이다.

```
1 void exit(int);
2 int getchar(void);
3 void *malloc(int);
4 int putchar(int);
5 int main1();
6 int main()
7 {
8     return main1();
9 }
10
11 char *my_realloc(char *old, int oldlen, int newlen)
12 {
13     char *new = malloc(newlen);
14     int i = 0;
15     while (i <= (oldlen - 1))
16     {
17         new[i] = old[i];
18         i = i + 1;
19     }
20
21     return new;
22 }
23
24 int nextc;
25 char *token;
26 int token_size;
27 void error()
28 {
29     exit(1);
30 }
31
32 int i;
33 void takechar()
34 {
35     if (token_size <= (i + 1))
36     {
37         int x = (i + 10) << 1;
38         token = my_realloc(token, token_size, x);
39         token_size = x;
40     }
41     token[i] = nextc;
42     i = i + 1;
43     nextc = getchar();
44 }
```

복원 방법: AST의 각 노드 종류(FuncDef, If, While, BinaryOp ...)를 다시 C 문법으로 바꿔 출력하면 된다. 본 보고서에서는 pycparser의 역변환(c_generator)으로 검증했고, if 개수·파라미터가 ast.c 분석 결과와 모두 일치함을 확인했다.

5. 배운 점 / 어려웠던 점

- **배운 점:** 코드를 텍스트가 아니라 트리(데이터)로 보면, 사람이 일일이 읽지 않아도 함수 개수·타입·조건문 같은 정보를 자동으로 뽑아낼 수 있다. 이것이 정적 분석/보안 도구의 기본 원리다.
- **어려웠던 점:**
 1. 타입이 `PtrDecl` → `TypeDecl` → `IdentifierType` 처럼 여러 겹으로 싸여 있어, 안쪽까지 **재귀로 따라 들어가야** `char *` 같은 최종 타입을 얻을 수 있었다.
 2. if 개수는 본문 어디에 중첩돼 있을지 몰라, 트리 전체를 **재귀 탐색**해서 세야 했다.
 3. (환경) 소스/터미널 인코딩(UTF-8 ↔ CP949) 불일치로 한글이 깨져, 전부 UTF-8로 통일했다.

6. 마지막 질문 (제출 요구 항목)

Q. AST(추상 구문 트리)는 실제 보안 분야에서 어떻게 활용될까? 특히 소스코드를 실행하지 않고도 취약점을 찾는 "정적 분석"에서 AST가 왜 핵심 도구로 쓰이는지, 그리고 AST 기반 분석의 한계(예: 런타임에만 드러나는 버그)는 무엇인지 궁금하다.

부록 — 챌린지(AST로 버그 패턴 탐지) 아이디어

AST를 순회하면서 특정 "위험 패턴"을 가진 노드를 찾으면 버그/취약점을 자동 탐지할 수 있다. 예:

- **if 안의 대입(=) 오용**: `If` → `cond` 가 `Assignment` 노드면 `==` 를 `=` 로 잘못 쓴 의심.
- **메모리 NULL 검사 누락**: `malloc` 호출 결과를 받은 변수에 대해 곧바로 `if(p==NULL)` 검사가 없는 경우.
- **gets, strcpy 등 위험 함수 호출**: `FuncCall` → `name` 이 위험 함수 목록에 있으면 경고.

`ast.c` 의 `count_if()` 처럼 트리를 재귀 순회하되, 조건을 "If 노드"에서 "위험 패턴 노드"로 바꾸면 그대로 간단한 취약점 스캐너가 된다.